

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И
ИНФОРМАТИКИ

Кафедра «Программная инженерия»

Лабораторная работа № N5
по дисциплине «Информационные технологии и программирование»
по теме: «Разработка веб-приложения на Spring Boot. Основы Spring Security»

Выполнил студент группы:
БПИ2401 Исламов Эмин Маратович
Проверил: Харрасов Камиль
Раисович

Москва 2026

Цель работы

Освоить аутентификацию и авторизацию в Spring Boot с ролями пользователей и stateless-подходом на JWT: регистрация, логин, выдача/проверка токена, защита эндпоинтов.

Архитектура и модули проекта

Слои (слоистая архитектура)

Controller - HTTP API (/auth, /users, /notifications).

Service - бизнес-логика регистрации/логина/CRUD.

Repository - доступ к БД через Spring Data JPA.

Security - фильтры, JWT-сервис, конфигурация Spring Security.

Пакеты (структура)

org.example.controller - AuthController/JwtAuthController, UserController, NotificationController

org.example.service - AuthService, UserService, NotificationService

org.example.repository - UserRepository, NotificationRepository

org.example.model.entity - User, Notification

org.example.model.dto - RegisterRequest, LoginRequest, UserDto, NotificationDto

org.example.model.enums - UserRole, NotificationStatus, NotificationChannel

org.example.security - JwtService, JwtAuthenticationFilter, CustomUserDetails, CustomUserDetailsService

org.example.config - SecurityConfig

Ход работы (с листингами кода)

Entity + роли пользователей

User (важные поля для security): email (unique), password, role.

```
@Getter
@Setter
@NoArgsConstructor
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false)
    private String name;
    @Column(nullable = false, unique = true)
    private String email;
    @Column(nullable = false)
    private String password;
    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private UserRole role;
    private String phone;
    private String deviceToken;
    private String telegramChatId;
    private LocalDateTime createdAt;
    @OneToMany(mappedBy = "recipient", cascade =
CascadeType.ALL)
```

```

        private List<Notification> notifications = new
        ArrayList<>();
    }

```

Репозиторий пользователей (поиск по email + уникальность)

UserRepository

```

@Repository
public interface UserRepository extends JpaRepository<User,
Long> {
    Optional<User> findByEmail(String email);
    boolean existsByEmail(String email);
}

```

CustomUserDetails + CustomUserDetailsService (под Spring Security)

CustomUserDetails

```

@AllArgsConstructor
public class CustomUserDetails implements UserDetails {
    private final User user;

    @Override
    public Collection<? extends GrantedAuthority>
    getAuthorities() {
        return List.of(new
        SimpleGrantedAuthority(user.getRole().name()));
    }

    @Override
    public String getPassword() {
        return user.getPassword();
    }
}

```

```

    }

    @Override
    public String getUsername() {
        return user.getEmail();
    }

    @Override
    public boolean isAccountNonExpired() {
        return true;
    }

    @Override
    public boolean isAccountNonLocked() {
        return true;
    }

    @Override
    public boolean isEnabled() {
        return true;
    }

    public User getUser() {
        return user;
    }
}

```

CustomUserDetailsService

```

@Service
@RequiredArgsConstructor
public class CustomUserDetailsService implements
UserDetailsService {

```

```

        private final UserRepository userRepository;

        @Override
        public UserDetails loadUserByUsername(String username)
            throws UsernameNotFoundException {

            User user =
                userRepository.findByEmail(username).orElseThrow(() -> new
                    UsernameNotFoundException(username));

            return new CustomUserDetails(user);
        }
    }
}

```

JWT сервис: генерация, извлечение username, проверка expiration

JwtService (минимально достаточный набор)

```

@Service
public class JwtService {

    private final String secret =
        "verySecretKeyForJwtTokenVerySecretKey12345";

    private final Key signingKey =
        Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));

    public String generateToken(String username) {

        SecretKey key =
            Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));

        return Jwts.builder()

            .setSubject(username)

            .setIssuedAt(new Date())

            .setExpiration(new
                Date(System.currentTimeMillis() + 1000 * 60

```

```

        * 60))
        .signWith(key, SignatureAlgorithm.HS256)
        .compact();
    }

    public String extractUsername(String token) {
        return extractAllClaims(token).getSubject();
    }

    private Claims extractAllClaims(String token) {
        return Jwts.parserBuilder()
            .setSigningKey(signingKey)
            .build()
            .parseClaimsJws(token)
            .getBody();
    }

    public boolean isTokenExpired(String token) {
        return
extractAllClaims(token).getExpiration().before(new Date());
    }

    public boolean isTokenValid(String token, UserDetails
userDetails) {
        return
extractUsername(token).equals(userDetails.getUsername()) &&
!isTokenExpired(token);
    }
}

```

JWT фильтр (OncePerRequestFilter)

JwtAuthenticationFilter - проверяем Bearer, вытаскиваем username, валидируем токен, кладем Authentication.

```
@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends
OncePerRequestFilter {

    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest
request,
                                HttpServletResponse
response,
                                FilterChain
filterChain) throws ServletException, IOException {

        String authHeader =
request.getHeader("Authorization");

        if (authHeader == null || !
authHeader.startsWith("Bearer ")) {

            filterChain.doFilter(request, response);
            return;
        }

        String token = authHeader.substring(7);

        String username =
jwtService.extractUsername(token);

        if (username != null &&
SecurityContextHolder.getContext().getAuthentication() ==
null) {
```



```

        UserDetails userDetails =
userDetailsService.loadUserByUsername(username);
        if (jwtService.isTokenValid(token,
userDetails)) {
            UsernamePasswordAuthenticationToken
authenticationToken =
                new
UsernamePasswordAuthenticationToken(
                    userDetails,
                    null,
userDetails.getAuthorities()
                );
            SecurityContextHolder.getContext().setAuthentication(authen
ticationToken);
        }
    }
    filterChain.doFilter(request, response);
}
}

```

SecurityConfig (полный переход на JWT)

Ключевое требование: STATELESS + addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class).

```

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {

```

```

        private final CustomUserDetailsService
customUserDetailsService;

        private final JwtAuthenticationFilter
jwtAuthenticationFilter;

        @Bean

        public PasswordEncoder passwordEncoder() {

            return new BCryptPasswordEncoder();

        }

        @Bean

        public DaoAuthenticationProvider
authenticationProvider() {

            DaoAuthenticationProvider provider = new
DaoAuthenticationProvider();

            provider.setUserDetailsService(customUserDetailsService);

            provider.setPasswordEncoder(passwordEncoder());

            return provider;

        }

        @Bean

        public AuthenticationManager
authenticationManager(AuthenticationConfiguration
configuration) throws Exception {

            return configuration.getAuthenticationManager();

        }

        @Bean

        public SecurityFilterChain
securityFilterChain(HttpSecurity http) throws Exception {

            http

                .csrf(csrf -> csrf.disable())

                .authorizeHttpRequests(auth -> auth

```

```

        .requestMatchers(HttpMethod.POST, "/auth/
register", "/auth/login").permitAll()
        .requestMatchers("/auth/**").permitAll()
        .requestMatchers("/error").permitAll()
        .requestMatchers("/admin/
**").hasRole("ADMIN")
        .requestMatchers("/users/
**").hasAnyRole("USER", "ADMIN")
        .requestMatchers("/notifications/
**").hasAnyRole("USER", "ADMIN")
        .anyRequest().authenticated()
    )
    .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELE
SS))
    .authenticationProvider(authenticationProvider(
))
    .addFilterBefore(jwtAuthenticationFilter,
UsernamePasswordAuthenticationFilter.class)
    .httpBasic(basic -> basic.disable())    // МОЖНО
ОСТАВИТЬ
    .formLogin(form -> form.disable());
    return http.build();
}
}

```

AuthController: register, register-admin, login

AuthController

```
@RestController
```

```

@RequiredArgsConstructor
@RequestMapping("/auth")
public class AuthController {
    private final AuthService authService;

    @PostMapping("/register")
    public String register(@RequestBody @Valid
RegisterRequest request) {
        authService.register(request);
        return "success";
    }

    @PostMapping("/register-admin")
    public String registerAdmin(@RequestBody @Valid
RegisterRequest request) {
        authService.registerAdmin(request);
        return "success";
    }
}

```

AuthService: уникальность email, BCrypt, обработка неверных логин/пароль

```

@Service
@RequiredArgsConstructor
public class AuthService {
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    public void register(RegisterRequest request) {
        if
(userRepository.existsByEmail(request.getEmail())) {

```

```

        throw new
ResponseStatusException(HttpStatus.CONFLICT, "Email already
exists");
    }

    User user = new User();
    user.setName(request.getName());
    user.setEmail(request.getEmail());
    user.setPassword(passwordEncoder.encode(request.getPassword
()));
    user.setRole(UserRole.ROLE_USER);
    user.setCreatedAt(LocalDateTime.now());
    userRepository.save(user);
}

public void registerAdmin(RegisterRequest request) {
    if
(userRepository.existsByEmail(request.getEmail())) {
        throw new
ResponseStatusException(HttpStatus.CONFLICT, "Email already
exists");
    }

    User user = new User();
    user.setName(request.getName());
    user.setEmail(request.getEmail());
    user.setPassword(passwordEncoder.encode(request.getPassword
()));
    user.setRole(UserRole.ROLE_ADMIN);
    user.setCreatedAt(LocalDateTime.now());
    userRepository.save(user);
}

```

```
}
```

Защита удаления пользователя через @PreAuthorize

UserController

```
@RestController
@RequestMapping("/users")
@RequiredArgsConstructor
public class UserController {

    private final UserService userService;
    private final UserMapper userMapper;

    @PostMapping("/add")
    public UserDto createUser(@RequestBody @Valid UserDto
request){

        User response = userService.createUser(request);
        return userMapper.toDto(response);
    }

    @GetMapping("/all")
    public List<UserDto> getAllUsers(){
        return userService.getAllUsers().stream()
            .map(userMapper::toDto)
            .toList();
    }

    @GetMapping("/{id}")
    public UserDto getUserById(@PathVariable("id") Long id)
{
        User response = userService.getUserById(id);
        return userMapper.toDto(response);
    }
}
```

```

    }

    @PutMapping("/{id}")
    public UserDto updateUser(@PathVariable("id") Long id,
                              @RequestBody @Valid UserDto
request){
        User response = userService.updateUser(id,
request);

        return userMapper.toDto(response);
    }

    @PreAuthorize("hasRole('ADMIN')")
    @DeleteMapping("/{id}")
    public String deleteUser(@PathVariable("id") Long id){
        userService.deleteUser(id);
        return "User deleted";
    }
}

```

Как что связано (логика выполнения)

1. Клиент отправляет JSON -> Controller принимает @RequestBody @Valid DTO.
2. Registration сохраняет пользователя в БД (пароль - BCrypt, роль - USER/ADMIN).
3. Login аутентифицирует через AuthenticationManager -> выдаёт JWT.
4. При запросах к защищённым URL клиент добавляет Authorization: Bearer <token>.
5. JwtAuthenticationFilter проверяет токен -> кладёт Authentication в SecurityContext.

6. SecurityFilterChain разрешает доступ согласно ролям и правилам.

Описание аннотаций

@RestController, @RequestMapping, @PostMapping/... - MVC слой.

@Service, @Repository - компоненты бизнес-логики/данных.

@RequiredArgsConstructor - Lombok: DI через final-поля.

@Entity, @Table, @Id, @GeneratedValue, @Column, @Enumerated - JPA маппинг.

@EnableMethodSecurity, @PreAuthorize - защита методов по ролям.

@Valid + @NotBlank/@Email/... - валидация входа, 400 при нарушении.

Проверка работы (что и где скринить)

1. 12.1 Register - POST /auth/register -> success

2. 12.4 Login - POST /auth/login -> токен

3. Проверка JWT:

* GET /users/all без header -> 401

* GET /users/all с Authorization: Bearer <token> -> 200

4. Роли:

* DELETE /users/{id} под USER -> 403

* то же под ADMIN -> 200

5. Уникальность email:

повторная регистрация тем же email -> 409 Conflict (скрин)

Вывод

В ходе лабораторной работы реализована система безопасности Spring Security с ролями и JWT: регистрация USER/ADMIN, проверка уникальности email, логин с выдачей токена, stateless-авторизация через фильтр JWT и защита критичных операций по роли администратора, при этом доступ к защищённым ресурсам осуществляется только при наличии валидного Bearer-токена.

Ответы на контрольные вопросы

1) Что такое аутентификация и чем она отличается от авторизации?

Аутентификация (authentication) - подтверждение личности: “кто ты?” (логин/пароль, токен).

Авторизация (authorization) - проверка прав: “что тебе можно?” (роли/permissions, доступ к URL/методам).

2) Что делает SecurityFilterChain?

SecurityFilterChain описывает набор security-фильтров и правил доступа, которые применяются к HTTP-запросам: какие URL открыты, какие требуют аутентификацию, какие роли нужны, какие механизмы входа включены (Basic, form login, JWT), как управлять сессиями, CSRF и т.д.

3) Зачем нужен UserDetailsService?

UserDetailsService - это точка интеграции Spring Security с твоими пользователями.

Он загружает пользователя по username (у тебя это email) и возвращает UserDetails: пароль (в виде хеша), роли/authorities, флаги активности. На этом строится проверка логина и выдача прав.

4) Для чего используется PasswordEncoder?

PasswordEncoder:

хеширует пароль при регистрации (encode)

сравнивает введенный пароль с хешем из БД при логине (matches)

Это делает проверку безопасной и корректной (например BCrypt хранит соль внутри хеша).

5) Почему пароли нельзя хранить в открытом виде?

Потому что при утечке БД:

злоумышленник получает пароли “как есть” и зайдет в аккаунты

люди повторно используют пароли на других сервисах -> компрометация везде

Хранить надо только хеш (BCrypt/Argon2), чтобы даже при утечке нельзя было легко восстановить пароль.

6) Какую роль выполняет AuthenticationManager?

AuthenticationManager - центральный механизм аутентификации.

Он принимает объект Authentication (например UsernamePasswordAuthenticationToken) и пытается его “проверить”, делегируя работу набору AuthenticationProvider (например DaoAuthenticationProvider). Возвращает “аутентифицированный” Authentication или бросает ошибку.

7) Что делает DaoAuthenticationProvider?

DaoAuthenticationProvider - провайдер аутентификации, который:

вызывает UserDetailsService.loadUserByUsername()

получает UserDetails

сравнивает пароль через PasswordEncoder

при успехе возвращает authenticated token с authorities

То есть это “классическая” проверка логин/пароль на основе БД.

8) Как работает HTTP Basic?

HTTP Basic - это схема, где клиент отправляет заголовок:

Authorization: Basic base64(username:password)

Сервер на каждом запросе пытается аутентифицировать пользователя по этим данным. Это stateless по природе, но не шифрует данные - безопасно только поверх HTTPS.

9) Что хранится в SecurityContext?

В SecurityContext хранится текущий Authentication, то есть:

кто пользователь (principal)

какие у него роли/authorities

authenticated ли он

SecurityContext - это “контекст безопасности” текущего запроса/сессии.

10) Как работает аутентификация через сессию?

При “сессионной” схеме:

1. пользователь логинится один раз
2. сервер создаёт HttpSession и сохраняет там SecurityContext
3. клиент получает cookie JSESSIONID
4. на следующих запросах cookie приходит -> сервер восстанавливает SecurityContext из сессии

То есть состояние логина хранится на сервере.

11) Что такое JWT и чем он отличается от сессии?

JWT - подписанный токен (обычно `header.payload.signature`), где в `payload` лежат `claims` (например `subject=email`, `exp`).

Отличия:

JWT: состояние “внутри токена”, сервер не хранит сессии (`stateless`), проверяется подпись + срок.

Сессия: состояние хранится на сервере, токен-идентификатор хранится у клиента (`cookie`), сервер по нему ищет сессию.

12) Зачем нужен JWT-фильтр?

JWT-фильтр нужен, чтобы до попадания в контроллер:

достать `Authorization: Bearer <token>`

распарсить токен, проверить подпись/`exp`

извлечь `username`

загрузить `UserDetails` и положить `Authentication` в `SecurityContext`

Без фильтра `Spring Security` “не узнает” пользователя из токена.

13) Что делает `addFilterBefore()` в конфигурации?

`addFilterBefore(A, B)` вставляет фильтр `A` перед стандартным фильтром `B` в цепочке.

В JWT-схеме важно поставить JWT-фильтр до фильтров, которые ожидают уже готовую аутентификацию, чтобы успеть заполнить `SecurityContext`.

14) В чем разница между `SessionCreationPolicy.IF_REQUIRED` и `STATELESS`?

IF_REQUIRED: сессия создаётся только если реально нужна (например для form login), может использоваться SecurityContext через JSESSIONID.

STATELESS: Spring Security не использует HttpSession для хранения контекста, каждый запрос должен сам приносить данные для аутентификации (JWT/Basic). Это стандарт для JWT.

15) Как проверить доступ к URL по роли пользователя?

Два основных способа:

A) В SecurityFilterChain (URL-level):

```
.authorizeHttpRequests(auth -> auth  
  
    .requestMatchers("/admin/**").hasRole("ADMIN")  
  
    .requestMatchers("/users/**").hasAnyRole("USER","ADMIN")  
  
    )
```

B) На методе (method-level):

```
@PreAuthorize("hasRole('ADMIN')")  
  
@DeleteMapping("/users/{id}")
```

Проверка в Postman:

логинишься как USER -> /admin/ping должен дать 403

логинишься как ADMIN -> /admin/ping должен дать 200